Praxis der Softwareentwicklung

Visualisierung von Verifikationsalgorithmen für Neuronale Netze

Praxis der Softwareentwicklung (PSE)
Wintersemester 2025/26

Testphase

Karlsruher Institut für Technologie (KIT)
KASTEL-Institut für Informationssicherheit und Verlässlichkeit
Anwendungsorientierte Formale Verifikation
Prof. Dr. Bernhard Beckert
Am Fasanengarten 5
Gebäude 50.34
76131 Karlsruhe

Betreuer: Philipp Kern

Projektteilnehmer:

Name	E-Mail-Adresse
Alexander Mikhaylov	uiqbn@student.kit.edu
Cedric Linde	unogx@student.kit.edu
Elias Dörr	uuxmf@student.kit.edu
Enrique Lopez	ukgyh@student.kit.edu
Paul Schepperle	ucnlo@student.kit.edu

Karlsruhe, 18. März 2026

Inhaltsverzeichnis

1 Einleitung **3**

2 Automatisierte Tests **4**
2.1 Unit Tests 4
2.2 Integrationstests 4
2.3 Testabdeckung 5
2.4 Regressionstests für Fehler 5
2.5 vnn-comp 2025 Tests 6

3 Manuelle Tests **9**
3.1 Belastungstests 9
3.2 Durchführung der Testszenarien 9
3.2.1 Testumgebung 9
3.2.2 Testszenario TS1 – Standardanalyse eines einzelnen Netzes 10
3.2.3 Testszenario TS2 – Vergleich zweier Netze 11
3.3 Systemtest 14
3.4 GUI 14
3.5 Qualität 17

4 Fehlerbehebung **18**
4.1 Bugtracking 18
4.2 Behobene Fehler 18

Glossary **20**

1 Einleitung

In dieser Phase stand die umfassende Überprüfung unserer Software im Mittelpunkt. Ziel war es, die Funktionsfähigkeit und Zuverlässigkeit des Programms systematisch zu validieren sowie mögliche Schwachstellen zu erkennen und zu beheben. Dabei konzentrierten wir uns sowohl auf kritische Fehler, die das Arbeitsergebnis der Anwendung beeinflussen könnten, als auch auf kleinere Mängel, die die Benutzerfreundlichkeit und das allgemeine Nutzungserlebnis beeinträchtigen.

Ein wesentlicher Bestandteil dieser Phase war die schrittweise Fehlersuche und Überarbeitung des bestehenden Programmcodes. Durch gezielte Tests und wiederholte Ausführung verschiedener Anwendungsfälle konnten wir die Stabilität der Software verbessern und ihre korrekte Funktionsweise sicherstellen. Darüber hinaus diente diese Phase dazu, die Qualität der Anwendung insgesamt zu erhöhen und eine verlässlichere Grundlage für die weitere Nutzung und Entwicklung zu schaffen.

Ergänzend dazu wurden Unit- und Integrationstests erstellt, mit denen sowohl einzelne Funktionen als auch das Zusammenspiel verschiedener Komponenten überprüft werden konnten. Diese Tests simulieren typische Abläufe innerhalb der Anwendung und ermöglichen eine strukturierte Kontrolle ausgewählter Funktionalitäten. Auf diese Weise konnten Fehler nicht nur identifiziert, sondern auch nachhaltig abgesichert werden, sodass bereits geprüfte Bereiche bei späteren Änderungen weiterhin korrekt funktionieren.

Insgesamt leistete die Testphase einen wichtigen Beitrag zur Qualitätssicherung des Projekts, da das fertige Softwareprodukt in dieser Phase gezielt validiert und getestet werden soll.

2 Automatisierte Tests

Zur Implementierung von automatisierten Tests haben wir die Python-Bibliothek `pytest` verwendet. Die Tests werden automatisch auf dem `master`-Branch von Github ausgeführt, sobald dieser geändert wird. Hierzu nutzen wir das CICD-System voin Github, sodass Entwickler bei jeder Änderung im Repository direkt Feedback bekommen.

2.1 Unit Tests

Unit Tests werden durchgeführt, um Klassen und Funktionen einer Software separat zu testen. Wenn Funktionen von anderen Klassen abhängen, bietet `pytest` die Möglichkeit, diese anderen Datenobjekte zu mocken. Dadurch wird die Signatur der Klassen kopiert und ein leeres Objekt erstellt, welches das echte Objekt imitiert, jedoch komplett vom Unit Test kontrolliert wird. Dadurch können auch Funktionen mit starken Abhängigkeiten isoliert werden.

Insgesamt haben wir 602 Unit Tests implementiert, die mit dem Command `pytest` ausgeführt werden können. Folgende Flags stehen dabei zur Verfügung:

- `-n=auto` führt Tests auf allen verfügbaren CPUs parallel aus.
- `-vnncomp-dir=<path>` führt die vnn-comp2025 Tests aus (siehe Sektion 2.5)
- `-ra` zeigt eine Übersicht mit mehr Informationen über übersprungene oder fehlgeschlagenen Tests an
- `-cov=nn_verification_visualisation -cov-report=html -cov-branch` erstellt eine ausführliche html-Übersicht über die Abdeckung

2.2 Integrationstests

Integrationstests prüfen, ob mehrere Teile der Anwendung in einem gemeinsamen Ablauf korrekt zusammenarbeiten. Dabei stehen vor allem Fehler an den Schnittstellen zwischen Benutzeraktionen, interner Logik und Zustandsänderungen im Fokus.

Unsere Integrationstests betreffen insbesondere Controller-Funktionen wie das Laden und Speichern von Bounds, das Löschen eines Netzwerks, das Erstellen eines Diagramms oder das

Starten von Samples. Gleichzeitig werden die zugehörigen Datenstrukturen und Zustände, etwa `NetworkVerificationConfig`, `InputBounds` und `DiagramConfig`, sowie die Zuordnung von Samples zu Bounds überprüft. Indirekt wird außerdem kontrolliert, ob über `request_autosave()` korrekt signalisiert wird, dass Änderungen gespeichert werden müssen. So helfen die Tests dabei, zentrale Abläufe der Anwendung abzusichern und Fehler nach Änderungen am Code frühzeitig zu erkennen.

2.3 Testabdeckung

Folgende Zeilenabdeckung wurde erreicht (aufgeteilt nach Subkomponenten):

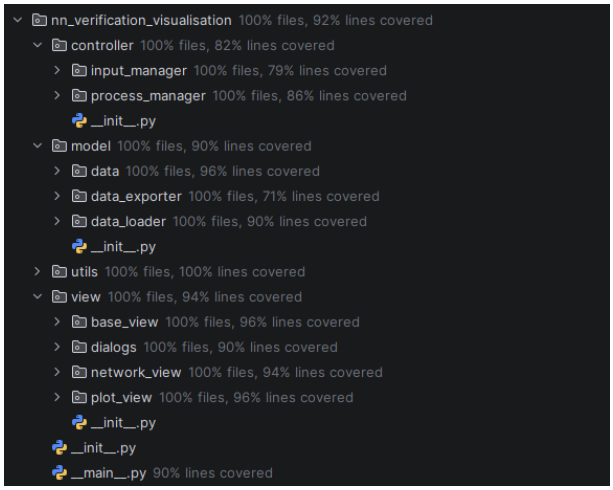

Abbildung 2.1: Zeilenabdeckung

Zudem wurde mithilfe von pytest ein ausführlicherer Abdeckungsbericht erstellt, der sowohl die Zeilenabdeckungen von einzelnen Dateien als auch die Verzweigungsabdeckung anzeigt.

2.4 Regressionstests für Fehler

Diese Regressionstests zu schreiben dient dazu, das Wiederauftreten von bereits behobenen Problemen zu verhindern. Dafür werden (wenn es sinnvoll ist) zum Beheben eines Fehler Unit-Tests hinzugefügt, die spezifisch testen, dass der Fehler nicht auftritt. Damit wird sichergestellt, dass:

- Die Fehlerkorrektur wirksam ist
- Der Fehler nicht in einer leicht abgeänderten Form wieder auftritt

Beispiel:
Ein im Verlauf der Testphase behobener Fehler war, dass bei aufeinanderfolgenden Layern mit jeweils einem Neuron im performanten Modus das Polygon eine Höhe von 0 hatte und dementsprechend keine Verbindung gezeichnet wurde (siehe Bug #113 auf Github).
Um die Lösung dieses Problems zu verifizieren, wurde in NetworkEdgeBatch ein Feld `"minimum_block_height"`definiert und der folgende Regressionstest geschrieben:

```
def test_block_has_minimum_height_when_single_node_per_layer(self, qapp):
   source = [make_node(0, 100)]
   target = [make_node(200, 100)]
   batch = NetworkEdgeBatch(source, target, force_block=True)
   assert batch.boundingRect().height() >= batch._NetworkEdgeBatch__minimum_block_height
```

Dieser Test prüft, dass bei jeweils einem Knoten pro Layer auf der gleichen Y-Position die Höhe des Blocks den definierten Mindestwert nicht unterschreitet. Dabei ist `make_node` eine Helfer-Methode, die ein QPoint-Objekt mit der gegebenen Position erstellt und zurückgibt.

2.5 vnn-comp 2025 Tests

Es wurden spezielle Testfälle erstellt, um folgende Eigenschaften großflächig für eine Menge an verfügbaren vnn-comp-benchmarks zu testen:

1. Das Netzwerk kann ohne einen Fehler geladen werden
2. Die Bounds können ohne einen Fehler geladen werden
3. Der auto_LiRPA box_ibp Algorithmus kann auf dem Netz ausgeführt werden (mit den ersten beiden Knoten des ersten Layers und 32 Richtungen als Eingabe)

Da es eine riesige Anzahl an Kombinationen aus .onnx-Netzen und zugehörigen .vnnlib-Bounds in den benchmarks gibt, werden nur eine bestimmte Anzahl an zufällig gewählten Kombinationen (die anhand der instances.csv Datei valide sind) getestet. Dabei kann über die Konstanten `_VNNCOMP_SAMPLE=1` und `_VNNCOMP_SEED=42` die Anzahl der Paare pro benchmark und der Seed für die zufällige Auswahl eingestellt werden.

Die Ergebnisse unserer Tests lassen sich auf der nächsten Seite finden.
Anmerkung: Es waren nicht alle benchmarks und von den benchmarks nicht alle bound-Dateien in unserem Test-Pool drin. Deshalb kann es sein, dass bei eigener Ausführung mit dem default-seed andere Ergebnisse herauskommen.

Benchmark	**Netzwerk**	**Fehler**	**Valide Netzstruktur**
acasxu_2023	ACASXU_run2a_4_2_batch_2000	-	J
cctsdb_yolo_2023	patch-1	Layer 0 has only 1 neuron	N
cersyve	lane_keep_pretrain_con	mat1 and mat2 shapes cannot be multiplied (1x4 and 6x3)	N
cgan_2023	cGAN_imgSz64_nCh_1	Bounds load failed: Missing bounds for X_5	N
cifar100_2024	CIFAR100_resnet_large	cannot reshape array of size 66 into shape (64,)	N
collins_rul_cnn_2022	NN_rul_small_window_20	Layer 0 has only 1 neuron	N
cora_2024	mnist-set	-	J
dist_shift_2023	mnist_concat	mat1 and mat2 shapes cannot be multiplied (1x200 and 794x200)	N
linearizenn_2024	AllInOne_120_50	Bounds load failed: Missing bounds for X_4	N
lsnc_relu	relu_quadrotor2d_state	mat1 and mat2 shapes cannot be multiplied (1x6 and 8x10)	N
malbeware	malware_malimg_family_scaled_linear-25	mat1 and mat2 shapes cannot be multiplied (1x25 and 4098x25)	J
ml4acopf_2024	14_ieee_ml4acopf	Bounds load failed: Missing bounds for X_22	N
nn4sys	pensieve_small_parallel	Bounds load failed: Missing bounds for X_96	N
relusplitter	mnist_fc_vnncomp2022_mnist-net_256x4_RSPLITTER... _3_0.05	-	J
sat_relu	unsat_v6_c30	Bounds load failed: Missing bounds for X_6	N
soundnessbench	model	Bounds load failed: Missing bounds for X_128	N
test	test_tiny	Layer 0 has only 1 neuron	J
tinyimagenet_2024	TinyImageNet_resnet_medium	cannot reshape array of size 66 into shape (64,)	N
tllverifybench_2023	tllBench_n=2_N=M=8_m=1_instance_0_1	Bounds load failed: Missing bounds for X_2	J
traffic_signs_recognition_2023	3_64_64_QConv_32_5_MP_2_BN_QConv_64_5_MP_2_BN_QConv_64_3_MP_2_BN_Dens...	cannot reshape array of size 3 into shape ()	N
vit_2023	pgd_2_3_16	cannot reshape array of size 50 into shape (48,)	N
yolo_2023	TinyYOLO	Bounds load failed: Missing bounds for X_8112	N

Tabelle 2.1: VNN-COMP Integration Test Results

Es gibt vier Abweichungen zwischen der Validität der Netzstruktur für unser Programm und dem Auftreten von Fehlern:

1. malbeware - In diesem Fall vertauscht Flatten die Eingabedimensionen, was von unserem Programm nicht unterstützt wird
2. test - Das test-tiny Netz hat nur einen Eingabeknoten, was die Bedeutsamkeit des Tests einschränken würde. In einem seperaten Test mit zwei Mal dem ersten Knoten als Eingabe hat test-tiny aber (im Gegensatz zu den anderen beiden Netzen mit diesem Fehler) funktioniert
3. tllverifybench - Das tllverifybench Netz hat als Eingabe `float32[unk__21,2]`, was nicht von unserem Programm unterstützt wird

Komplett akzeptiert wurden nur die Netze von den acasxu_2023 und relusplitter benchmarks. Es trat jedoch bei keinem Ladevorgang eines Netzes ein Fehler auf. Wenn es Fehler bei dem Laden von bounds gab, dann nur, weil aufgrund von nicht unterstützten Struktureigenschaften die Eingabe falsch interpretiert und deshalb eine andere .vnnlib Datei gefordert wurde. Alle anderen Fehler kamen vom Umformen des Netzes oder vom auto_LiRPA box_ibp Algorithmus, wo sich eine unpassende Struktur spätestens bemerkbar macht.

3 Manuelle Tests

Zusätzlich zu den automatisierten Unit-Tests haben wir verschiedene manuelle Tests durchgeführt, um die Funktionalität der Benutzeroberfläche, die Belastbarkeit des Systems und das Zusammenspiel verschiedener Systemkomponenten zu testen. Die in diesem Kapitel durchgeführten Test- und Ergebnisdateien sind alle in dem Github Repository des Projekts zu finden.

3.1 Belastungstests

Für einen Belastungstest wurde ein Netzwerk mit 75.000.000 Kanten benutzt. Das Netz ließ sich laden und es konnten Bounds hinzugefügt werden. Auch das Auswählen eines Neuronenpaares und eines Algorithmus hat funktioniert. Die Berechnung wurde gestartet und nach ca. 3 Minuten abgebrochen. Das Abbrechen hat fehlerfrei funktioniert und der Arbeitsspeicher wurde wieder frei gegeben.

3.2 Durchführung der Testszenarien

Die in Abschnitt 10.3 des Pflichtenhefts definierten Testszenarien wurden durchgeführt. Ziel war es zu überprüfen, ob die implementierten Funktionen den erwarteten Eigenschaften entsprechen und ob die Anwendung stabil unter realistischen Nutzungsbedingungen arbeitet.

3.2.1 Testumgebung

Alle Tests wurden auf folgendem System durchgeführt:

- Betriebssystem: macOS Sonoma 14.6.1
- Prozessor: 2,4 GHz Quad-Core Intel Core i5
- Arbeitsspeicher: 8 GB 2133 MHz LPDDR3
- Grafikhardware: Intel Iris Plus Graphics 655 (1536 MB)

3.2.2 Testszenario TS1 – Standardanalyse eines einzelnen Netzes

Im ersten Testszenario wurde ein typischer Analyseablauf eines einzelnen neuronalen Netzes durchgeführt. Dabei wurde überprüft, ob ein Netz geladen, Neuronen ausgewählt und Visualisierungen mit verschiedenen Algorithmen erstellt werden können.

Konfiguration

- Geladenes Netz: NN1
- Geladene Bounds: B1
- Algorithmus: `box_ibp_numpy.py`

Durchführung

- Das Standardnetz NN1 wurde erfolgreich geladen.
- Die Bounds B1 wurden geladen.
- Ein Sample-Run wurde erfolgreich durchgeführt.
- Navigation in der Netzansicht funktionierte erwartungsgemäß:
  - Zoom funktioniert
  - Pan funktioniert
- Wechsel zwischen Netzansicht und Diagrammansicht funktioniert.

Anschließend wurden zwei Neuronen ausgewählt:

- Layer 3, Neuron 4
- Layer 3, Neuron 8

Daraufhin wurde eine Visualisierung mit dem Algorithmus `box_ibp_numpy.py` erzeugt.

Zusätzlich wurde der Zonotop-Algorithmus erfolgreich ausgeführt.

Persistenztest

Nach einem Neustart der Anwendung waren weiterhin vorhanden:

- geladenes Netz
- geladene Bounds
- erzeugtes Diagramm

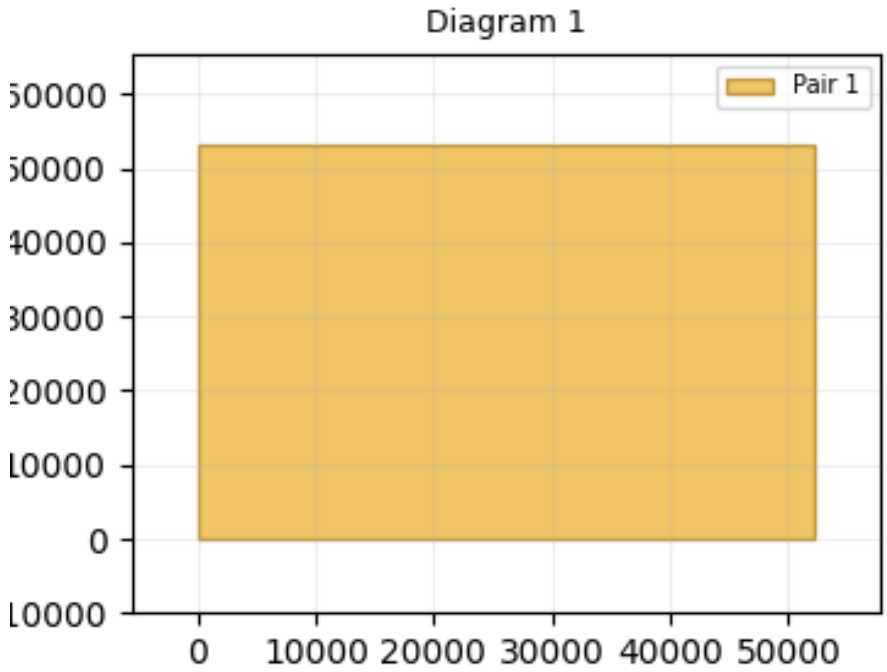

Abbildung 3.1: Ergebnis der Visualisierung im Testszenario TS1

Damit funktioniert das Wiederherstellen einer Session wie erwartet.

Nicht implementierte Soll/Kann-Funktionen

Folgende optionale Funktionen aus dem Pflichtenheft sind derzeit nicht vorhanden:

- 3-dimensionale Visualisierung
- n-dimensionale Visualisierung

Diese gehören zu optionalen Erweiterungen und beeinträchtigen die Muss-Kriterien nicht.

Ergebnis

Alle erwarteten Eigenschaften bezüglich der implementierten Funktionen sind erfüllt.

3.2.3 Testszenario TS2 – Vergleich zweier Netze

Im zweiten Testszenario wurde die Anwendung unter einem realistischeren Setup mit zwei Netzen getestet.

Konfiguration

- Geladene Netze: NN2 (zwei Netze gleichzeitig)
- Bounds: B1

Die im Pflichtenheft vorgesehenen CPU- und RAM-Grenzen wurden in der aktuellen Implementierung nicht umgesetzt.

Durchführung

In beiden Netzen wurden vergleichbare Neuronen ausgewählt:

- Layer 1, Neuron 100
- Layer 1, Neuron 200

Es wurden zwei verschiedene Algorithmen parallel ausgeführt:

- Box IBP (NumPy-Implementierung)
- Simple Zonotope Algorithmus

Beide Berechnungen wurden gleichzeitig gestartet.

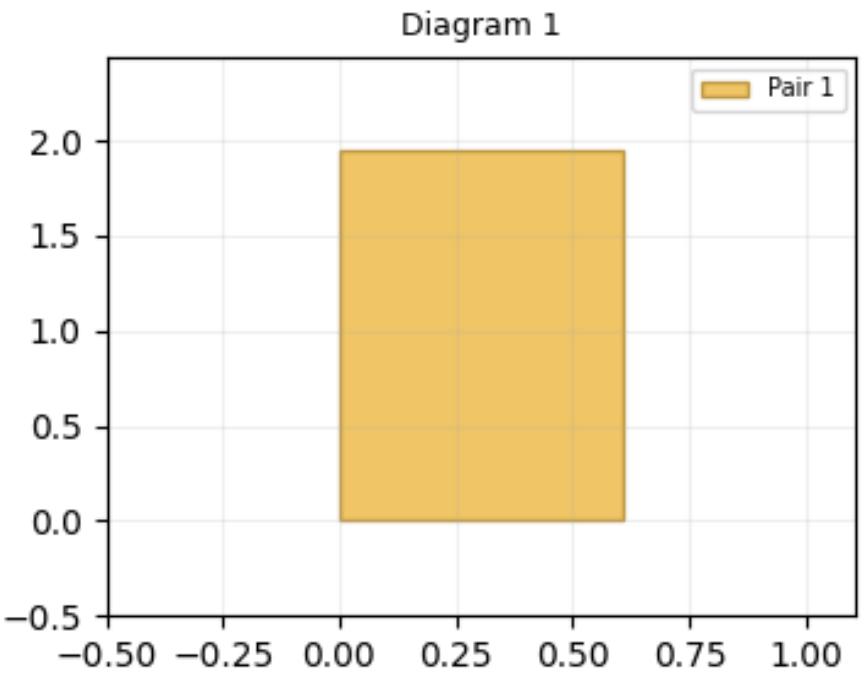

Abbildung 3.2: Erste Visualisierung im Vergleichs-Testszenario TS2

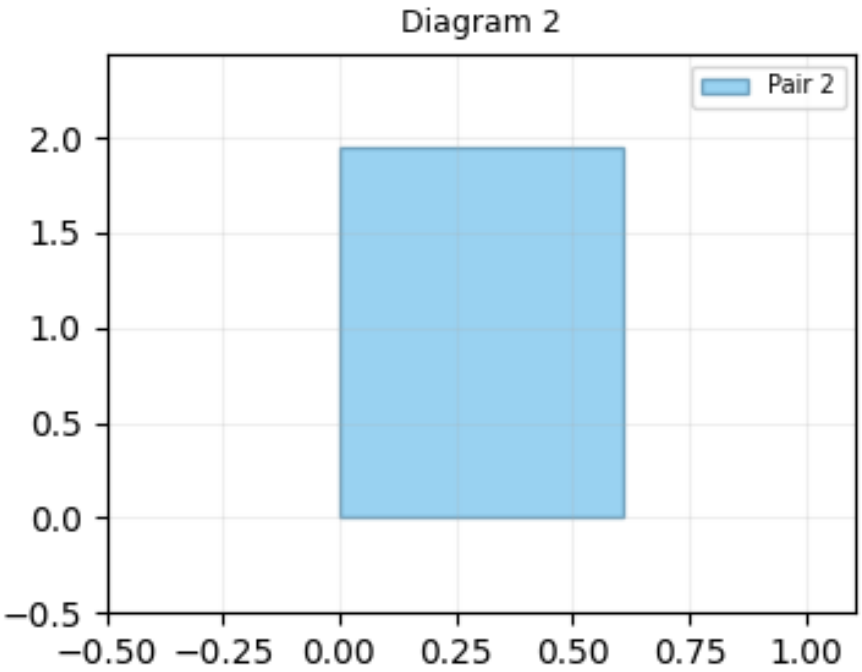

Abbildung 3.3: Zweite Visualisierung im Vergleichs-Testszenario TS2

Abbruch eines Algorithmus

Der Testalgorithmus `cancel_test.py` wurde gestartet und anschließend manuell abgebrochen. Die Berechnung wurde korrekt beendet und die Anwendung blieb stabil.

Fehlerhafte Eingaben

Mehrere ungültige und fehlerhafte Eingabedateien wurden getestet:

- Ungültige Datentypen können in keiner Input-Option geladen werden.
- fehlerhafte Datei `NNE.onnx`: wird nicht geladen, allerdings erfolgt aktuell keine Fehlermeldung.
- fehlerhafte Datei `BE1.csv` erzeugt beim Laden folgenden Error:

ValueError('Every index in the csv file has to appear exactly once and be in range 0..N-1.')

- fehlerhafte Datei `BE2.vnnlib` erzeugt beim Laden folgenden Error::

ValueError('Invalid expression.')

Ergebnis

Alle implementierten Eigenschaften funktionierten erwartungsgemäß und die Anwendung blieb stabil während der gesamten Tests.

3.3 Systemtest

Systemtest: Getestete Umgebungen:

- macOS 14: funktioniert
- macOS 15: funktioniert
- Ubuntu 24: funktioniert
- Windows 11: funktioniert
- Fedora 43: funktioniert

Ergebnis: Das Programm ist auf allen getesteten Systemen lauffähig.

3.4 GUI

Für das Testen der GUI wurden verschiedene Befehle ausgeführt und auf Korrektheit geprüft. Außerdem wurde die Geschwindigkeit und Memoryusage für das Starten der GUI und das Laden von Netzen getestet. Die hier benutzten Netzwerke hatten folgende Eigenschaften:

- `simple_3_layer_x10`: 140 Neuronen, 4.800 Verbindungen
- `simple_3_layer_x50`: 600 Neuronen, 80.000 Verbindungen
- `simple_3_layer_x100`: 1.400 Neuronen, 480.000 Verbindungen
- `NN1`: 2.006 Neuronen, 753.000 Verbindungen

NN1 lädt in der GUI trotz mehr Verbindungen schneller, weil die Ladezeit nicht nur von der Anzahl der Kanten abhängt. Entscheidend sind auch die Netzstruktur, die Verteilung der Layergrößen sowie der Aufwand für ONNX- Validierung, Szenenaufbau und Darstellung in der GUI. Die Messungen zeigen daher, dass breite synthetische Netze wie simple_3_layer_x100 beim Laden und Anzeigen teurer sein können als NN1, obwohl NN1 insgesamt mehr Verbindungen besitzt.

GUI Smoke Test Summary

Each executed check is shown with its final status.

Window Init	passed
Window Layout	passed
View Switching	passed
Tab Container Present	passed
Network Load Real File	passed
Bounds Import Real File	passed
Bounds Save Via Ui	passed
Bounds Edit And Remove Via Ui	passed
Diagram Tab Creation	passed
Diagram Selection State	passed
Plot Tab Close Removes Storage	passed

Abbildung 3.4: Funktionale Tests der GUI

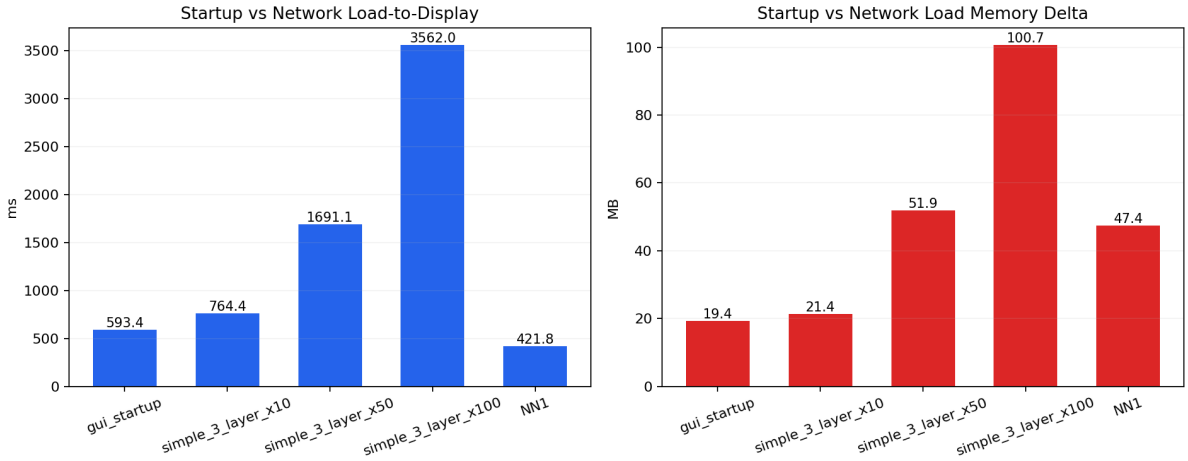

Abbildung 3.5: Nichtfunktionale Tests der GUI

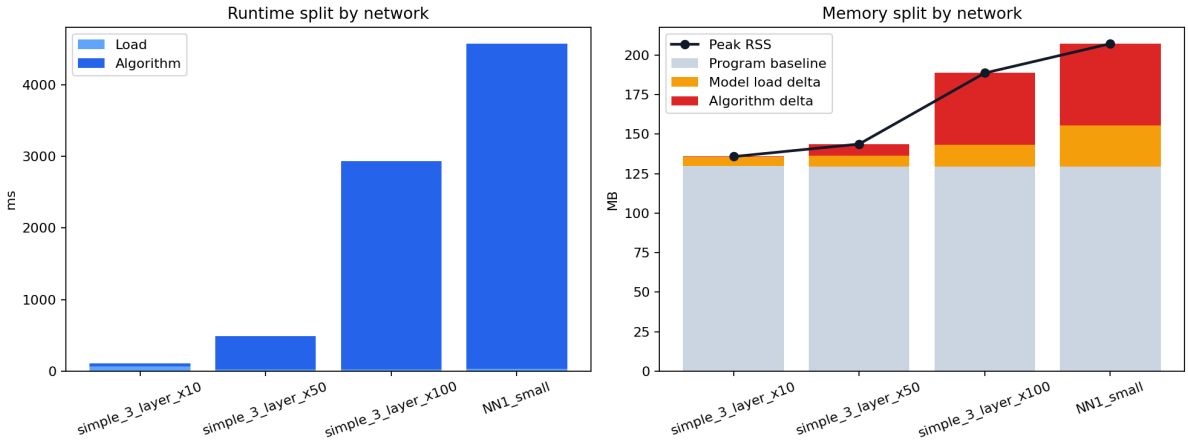

Abbildung 3.6: Geschwindigkeits- und Speichertests

3.5 Qualität

Zur Kontrolle der Qualität der erstellbaren Diagramme wurden verschiedene Tests ausgeführt. Der erste Test hat die Geschwindigkeit und Memoryusage bei der Ausführung von einem Algorithmus (Box IBP NumPy) auf verschiedenen Netzen ohne GUI ermittelt. Danach wurde die Genauigkeit der Ergebnisse getestet, indem die Überapproximation mit Testsamples verglichen wurde. Hierbei wurden verschiedene Netzwerke mit verschiedenen Bounds getestet.

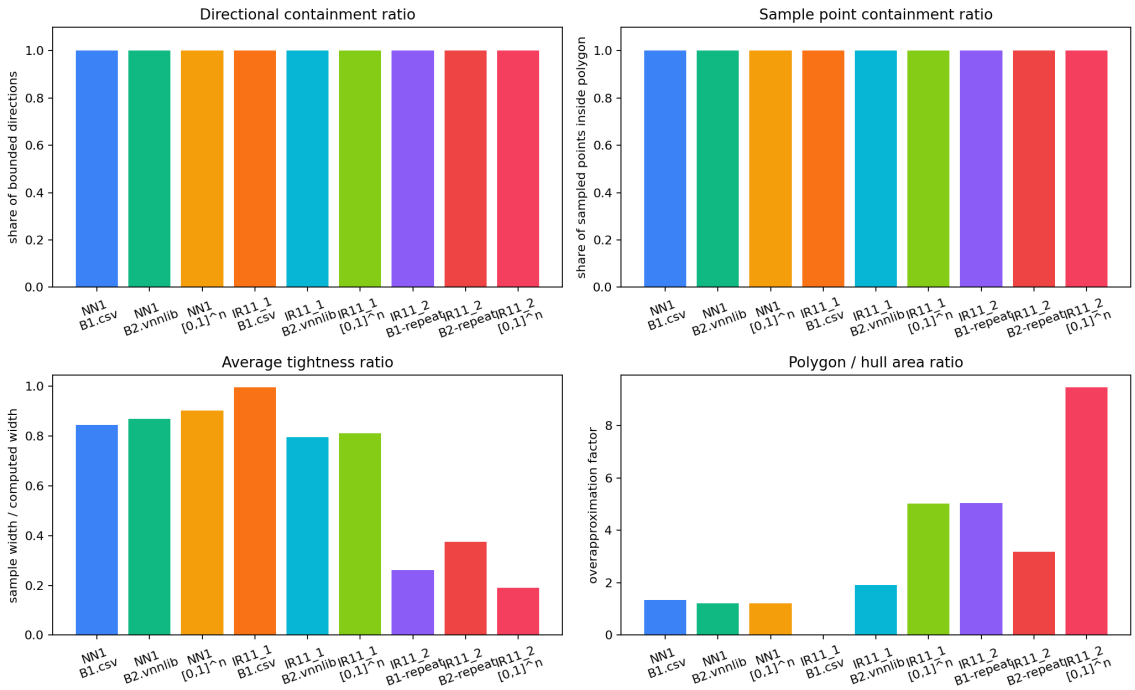

Abbildung 3.7: Genauigkeitstest

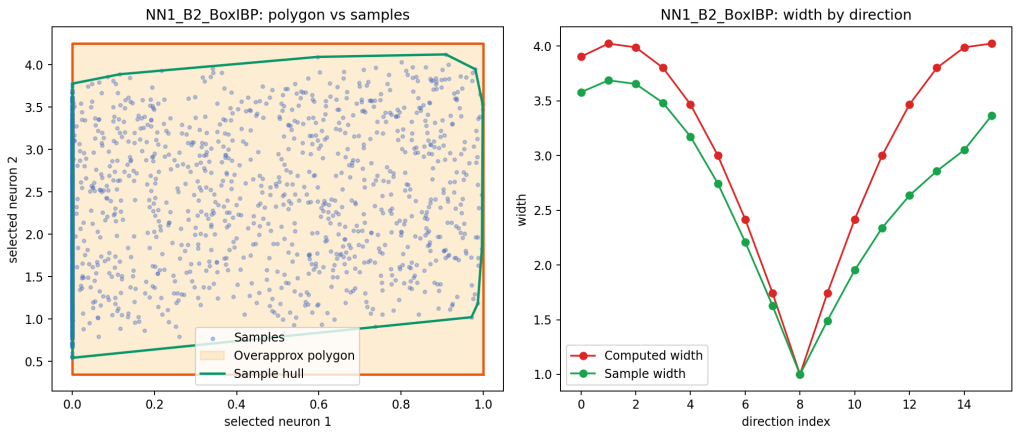

Abbildung 3.8: Beispiel eines Genauigkeitstests

4 Fehlerbehebung

4.1 Bugtracking

Zum Verwalten der gefundenen Fehler haben wir das Issue-System von Github benutzt. Hier können sowohl wir als auch externe Nutzer des Programms Fehlerberichte erstellen. Jedes Issue folgt einem festen Schema, welches aus einer Beschreibung, einer Anleitung zum Reproduzieren des Bugs und ggf. Screenshots besteht. Außerdem sind Zuweisungen an bestimmte Mitglieder des Projektes möglich.
Issues können von Mitgliedern geschlossen werden, wobei ein Erfolgsstatus übergeben wird. Außerdem können Issues mit Pull-Requests verknüpft werden, sodass genau ersichtlich ist, wann ein bestimmter Bug behoben wurde.

4.2 Behobene Fehler

1. Ein großer Fehler war, dass das Programm zuerst nur mit Testnetzwerken funktionerte, die wir selbst gemacht hatten und nicht auf den vorgegebenen Testnetzwerken der Website "VNN Comp". Hierzu wurden Anpassungen im Programm gemacht. Das erste Problem war, dass diese Netzwerke einen Preprocessingschritt hatten, welcher falsch erkannt wurde. Deswegen wurde nun ein Offset eingabaut, der überprüft, ob der 1. Layer bereits ein Matrixlayer ist oder ein Preprocessinglayer. Das zweite Problem war, dass die Daten in einem anderen Format gespeichert waren als die Daten in unseren ersten Testnetzwerken. Dies wurde behoben, indem vor der Modifizierung des Netzwerks alle Initializer des Netzwerks erst gelöscht und dann neu beschrieben worden sind, so haben sie alle das gleiche Format.
2. Das UI wurde an vielen Stellen optimiert. Bei Netzwerken mit großem Inputlayer war es zuvor umständlich neue Bounds zu erstellen, da die gesamte Seitenleiste im `NetworkTab` scrollable war. Jetzt ist nur noch die Bounds-Liste Teil der Scrollbar, sodass alle Buttons außerhalb der Liste zu jedem Zeitpunkt sichtbar sind.
   Außerdem war der Wechsel zwischen NetworkView und PlotView durch das Theme-Update mit größerem Lag verbunden. Durch Optimierung unseres Styles konnte diese Zeit etwa halbiert werden.

Außerdem wurden am Theme einige Optimierungen vorgenommen, um die Lesbarkeit von Texten zu erhöhen und das Programm einheitlicher zu gestalten.

3. Unsere Releases, die wir zuvor mit `pyinstaller` erstellt haben, waren nicht in der Lage Algorithmen auszuführen, die externe Bibliotheken benötigen. Deshalb haben wir uns schlussendlich dazu entschieden, stattdessen unseren Quellcode mit eigenem Installer-Script als Paket zu veröffentlichen. Der Installer erstellt lokal ein neues `venv` (eine neue virtuelle Pythonumgebung), in der alle relevanten Pakete installiert werden. Der Nutzer kann dann ggf. eigene Bibliotheken hinzufügen, die von seinen Algorithmen benötigt werden. Auf Github ist dokumentiert, wie der Installationsprozess genau abläuft.

Glossar

.csv Dateiformat zum Verwalten von Text- und Zahlenwerten in Form von Tabellen. Wird im Programm für den Import und Export der Bounds eines Netzes benutzt.

.onnx Open Neural Network Exchange, quelloffenes Dateiformat zum Speichern von Neuronalen Netzen. Kann zum Auswerten importiert werden..

.py Dateiformat eines Pythonskriptes. Kann genutzt werden, um eigene Algorithmen in die Software einzubinden.

.svg Dateiformat für Vektorgrafiken. Bilder werden nicht als als einzelne Pixel gespeichert, sondern als mathematisch definierte Objekte wie Linien, Kreise und Kurven.

.vnnlib Beinhaltet die Input- und Output Bounds eines Netzwerks, nicht aber das Netzwerk selbst.

Bibliothek Ansammlung an Softwarecode anderer Entwickler, die diesen als Paket zur Einbindung in fremde Projekte zur Verfügung stellen..

Binding Verbindung zwischen zwei Softwarekomponenten, bspw. zwischen dem Programm und einer in einer anderen Sprache geschriebene Bibliothek..

Caching Die Speicherung zuvor geladener oder berechneter Daten im Zwischenspeicher, ermöglicht einen schnellen erneuten Zugriff auf diese.

Code Coverage Tool Tool zur Erfassung davon, wie viel des Codes durch Tests abgedeckt ist.

deterministischer Algorithmus Algorithmus, der bei gleicher Eingabe immer zum gleichen Ergebnis kommt.

Diagramm Stellt den Zusammenhang eines Neuronenpaares mit einem Algorithmus dar. Kann entweder als einzelner Tab oder als Teil eines Vergleiches angezeigt werden.

draggen Bewegen der Maus während ein Button gedrückt gehalten wird. Geste, um UI-Elemente zu verschieben oder im dreidimensionalen Raum zu drehen..

Dropdown-Menü Ein UI-Element zum Auswählen eines Entrags aus einer vorgegebenen Liste an Optionen, wird zur Auswahl von Neuronalen Netzen und Algorithmen benutzt.

einsatzbereites Netz Ein Netz, bei dem Eingabebounds schon geladen/eingestellt wurden.

großes Netz Ein Netz mit mindestens 10000 Knoten.

GUI Graphical User Interface - Die Benutzeroberfläche des Programms.

Layer Einzelne Schicht eines neuronalen Netzes. Es gibt einen Input- und einen Outputlayer für die Ein- und Ausgabe des Netzes.

Log-Datei Datei zur Speicherung von Systemnachrichten.

MemoryError Ein speicherbedingter Fehler.

Neuron Ein einzelner Knoten eines Netzwerkes. Wird über gewichtete Kanten mit anderen Neuronen verbunden.

Neuronales Netz Netz aus künstlichen Neuronen, an biologischen Gehirnen inspiriert. Besteht aus Nodes in Layern und gewichteten Verbindungen.

Open-Source-Lizenz Eine Lizenz für quelloffene Software.

pan Bewegen eines UI-Elements durch Draggen der Maus in zwei Dimensionen. Wird für die Navigation von Diagrammen und Neuronalen Netzen benutzt.

PEP 8 Ein Style Guide für Python Code.

Polytop Eine Verallgemeinerung von konvexen Vielecken in beliebiger Dimension.

Session Engl. für Sitzung, beschreibt die Zeitperiode ab dem Öffnen des Programms durch einen Nutzer bis zum vollständigen Schließen..

Standartnetz Ein Netz mit 2000 Neuronen, die auf 4 Layer verteilt sind.

System Usability Scale Tool zur Erfassung der subjektiv empfundenen Usability.

Tab Ein "karteikartenartiger" Reiter in einer Auflistung, wird zum Wechseln der Ansicht zwischen verschiedenen Neuronalen Netzen, Diagrammen und Vergleichen verwendet.

Unit Test Test zur Überprüfung eines klar abgegrenzten Teils des Programms.

Vergleich Ein Vergleich zeigt Diagramme mehrerer Neuronenpaare.

Zonotop Ein Zonotop ist die Minkowski-Summe einer endlichen Anzahl an Vektoren. Die enstandene Fläche ist konvex. Es wird im Diagramm dargestellt und aus neuronalen Netzen generiert.

zoom Ändern der Anzeigegröße eines UI-Elements. Wird für die Navigation von Diagrammen und Neuronalen Netzen benutzt.